Оптимизация числа запусков тестов

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$
- 40 тестов, запускаемых при проверке релиза; $(j = 1, \ldots, 40)$

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$
- 40 тестов, запускаемых при проверке релиза; $(j = 1, \ldots, 40)$
- каждый тест запускался 5 раз на каждом релизе; $(k = 1, \ldots, 5)$

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$
- 40 тестов, запускаемых при проверке релиза; $(j = 1, \ldots, 40)$
- каждый тест запускался 5 раз на каждом релизе; $(k = 1, \ldots, 5)$
- $I_{jk}^{(i)} \in \{0, 1\}$ — результат запуска k-ой итерации j-го теста i-го релиза.

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$
- 40 тестов, запускаемых при проверке релиза; $(j = 1, \ldots, 40)$
- каждый тест запускался 5 раз на каждом релизе; $(k = 1, \ldots, 5)$
- $I_{jk}^{(i)} \in \{0, 1\}$ — результат запуска k-ой итерации j-го теста i-го релиза.
  - $I_{jk}^{(i)} = 1$: тест обнаружил проблему (`red_flag`);

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$
- 40 тестов, запускаемых при проверке релиза; $(j = 1, \ldots, 40)$
- каждый тест запускался 5 раз на каждом релизе; $(k = 1, \ldots, 5)$
- $I_{jk}^{(i)} \in \{0, 1\}$ — результат запуска k-ой итерации j-го теста i-го релиза.
  - $I_{jk}^{(i)} = 1$: тест обнаружил проблему (`red_flag`);
  - $I_{jk}^{(i)} = 0$: проблема не обнаружена.

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$
- 40 тестов, запускаемых при проверке релиза; $(j = 1, \ldots, 40)$
- каждый тест запускался 5 раз на каждом релизе; $(k = 1, \ldots, 5)$
- $I_{jk}^{(i)} \in \{0, 1\}$ — результат запуска k-ой итерации j-го теста i-го релиза.
  - $I_{jk}^{(i)} = 1$: тест обнаружил проблему (`red_flag`);
  - $I_{jk}^{(i)} = 0$: проблема не обнаружена.

1. Постановка задачи

Исходные данные

- 70 релизов в исторических данных; $(i = 1, \ldots, 70)$
- 40 тестов, запускаемых при проверке релиза; $(j = 1, \ldots, 40)$
- каждый тест запускался 5 раз на каждом релизе; $(k = 1, \ldots, 5)$
- $I_{jk}^{(i)} \in \{0, 1\}$ — результат запуска k-ой итерации j-го теста i-го релиза.
  - $I_{jk}^{(i)} = 1$: тест обнаружил проблему (`red_flag`);
  - $I_{jk}^{(i)} = 0$: проблема не обнаружена.

Цель

Для каждого теста выбрать число запусков n_j, минимизируя их суммарную стоимость при условии, что вероятность выпустить релиз с невыявленной проблемой не превышает 5%.

2. Математическая модель

Для каждого релиза i и теста j вводим скрытую переменную:

$$\theta_j^{(i)} \sim \text{Bernoulli}(q_j),$$

где $\theta_j^{(i)} = 1$ означает, что тестируемый компонент действительно имеет проблему.

2. Математическая модель

Для каждого релиза i и теста j вводим скрытую переменную:

$$\theta_j^{(i)} \sim \text{Bernoulli}(q_j),$$

где $\theta_j^{(i)} = 1$ означает, что тестируемый компонент действительно имеет проблему.

Если проблема есть, каждый запуск может обнаружить её:

$$(I_{jk}^{(i)} | \theta_j^{(i)} = 1) \sim \text{Bernoulli}(p_j).$$

2. Математическая модель

Для каждого релиза i и теста j вводим скрытую переменную:

$$\theta_j^{(i)} \sim \text{Bernoulli}(q_j),$$

где $\theta_j^{(i)} = 1$ означает, что тестируемый компонент действительно имеет проблему.

Если проблема есть, каждый запуск может обнаружить её:

$$(I_{jk}^{(i)} | \theta_j^{(i)} = 1) \sim \text{Bernoulli}(p_j).$$

При этом мы считаем, что:

$$(I_{jk}^{(i)} | \theta_j^{(i)} = 0) = 0.$$

3. Оценка параметров: функция правдоподобия

Написали логарифм правдоподобия:

$$\ln L = \sum_{i,j} \ln \left(q_j \cdot p_j^{\sum_{k=1}^{5} x_{jk}^{(i)}} \cdot (1-p_j)^{5-\sum_{k=1}^{5} x_{jk}^{(i)}} + (1-q_j) \cdot I\{x_{j1}^{(i)} = \cdots = x_{jk}^{(i)} = 0\} \right).$$

3. Оценка параметров: функция правдоподобия

Написали логарифм правдоподобия:

$$\ln L = \sum_{i,j} \ln \left(q_j \cdot p_j^{\sum_{k=1}^{5} x_{jk}^{(i)}} \cdot (1-p_j)^{5-\sum_{k=1}^{5} x_{jk}^{(i)}} + (1-q_j) \cdot I\{x_{j1}^{(i)} = \cdots = x_{jk}^{(i)} = 0\} \right).$$

Практическая реализация

Для каждого теста максимизируем логарифм правдоподобия численно с ограничениями $0 \leq q_j, p_j \leq 1$.

3. Оценка параметров: функция правдоподобия

Написали логарифм правдоподобия:

$$\ln L = \sum_{i,j} \ln \left(q_j \cdot p_j^{\sum_{k=1}^{5} x_{jk}^{(i)}} \cdot (1-p_j)^{5-\sum_{k=1}^{5} x_{jk}^{(i)}} + (1-q_j) \cdot I\{x_{j1}^{(i)} = \cdots = x_{jk}^{(i)} = 0\} \right).$$

Практическая реализация

Для каждого теста максимизируем логарифм правдоподобия численно с ограничениями $0 \le q_j, p_j \le 1$.

Важный частный случай

Если за все 70 релизов не было ни одного `red_flag`, параметры q_j и p_j по этим данным нельзя оценить раздельно: максимум достигается, например, при $q_j = 0$ или при $p_j = 0$. Для таких тестов нужна отдельная договорённость или дополнительная информация.

Альтернативный подход

Теоретическая вероятность обнаружить ошибку:

$$r_j = q_j \left(1 - (1 - p_j)^5\right).$$

Альтернативный подход

Теоретическая вероятность обнаружить ошибку:

$$r_j = q_j \left(1 - (1 - p_j)^5\right).$$

Эмпирическая оценка вероятности обнаружения:

$$\hat{r}_j = \frac{1}{70} \sum_{i=1}^{70} I\{S_j^{(i)} > 0\}.$$

Альтернативный подход

Теоретическая вероятность обнаружить ошибку:

$$r_j = q_j \left(1 - (1 - p_j)^5\right).$$

Эмпирическая оценка вероятности обнаружения:

$$\hat{r}_j = \frac{1}{70} \sum_{i=1}^{70} I\{S_j^{(i)} > 0\}.$$

Где количество срабатываний в i-ом релизе:

$$S_j^{(i)} = \sum_{k=1}^{5} I_{j,k}^{(i)}.$$

Условное правдоподобие

Строим функцию условного правдоподобия для тех случаев, когда мы точно знаем, что ошибка есть (отсекаем нули).

$$L = \prod_{i|s_j^{(i)}>0} \frac{C_5^{s_j^{(i)}} p_j^{s_j^{(i)}} (1-p_j)^{5-s_j^{(i)}}}{1-(1-p_j)^5} \Longrightarrow \text{оценка } \hat{p}_j.$$

Условное правдоподобие

Строим функцию условного правдоподобия для тех случаев, когда мы точно знаем, что ошибка есть (отсекаем нули).

$$L = \prod_{i|S_j^{(i)}>0} \frac{C_5^{S_j^{(i)}} p_j^{S_j^{(i)}} (1-p_j)^{5-S_j^{(i)}}}{1-(1-p_j)^5} \Longrightarrow \text{оценка } \hat{p}_j.$$

Оценка $\hat{q}_j$ выражается из базовой формулы подстановкой полученных оценок $\hat{p}_j$ и $\hat{r}_j$:

$$\hat{q}_j = \frac{\hat{r}_j}{1-(1-\hat{p}_j)^5}.$$

Условное правдоподобие

Строим функцию условного правдоподобия для тех случаев, когда мы точно знаем, что ошибка есть (отсекаем нули).

$$L = \prod_{i|s_j^{(i)}>0} \frac{C_5^{s_j^{(i)}} p_j^{s_j^{(i)}} (1-p_j)^{5-s_j^{(i)}}}{1-(1-p_j)^5} \Longrightarrow \text{оценка } \hat{p}_j.$$

Оценка $\hat{q}_j$ выражается из базовой формулы подстановкой полученных оценок $\hat{p}_j$ и $\hat{r}_j$:

$$\hat{q}_j = \frac{\hat{r}_j}{1-(1-\hat{p}_j)^5}.$$

Замечание

Сравнив оценки полученные двумя подходами пришли к выводу, что они почти не отличаются (погрешность порядка 10^{-4}).

4. Ограничения

Требование бизнеса

Вероятность выпуска проблемного релиза при условии наличия реальной проблемы должна быть не более 5%:

$$P(\forall j \; \forall k \leq n_j \; I_{j,k}^{(i)} = 0 \mid \exists j : \theta_j^{(i)} = 1) \leq 0.05.$$

4. Ограничения

Требование бизнеса

Вероятность выпуска проблемного релиза при условии наличия реальной проблемы должна быть не более 5%:

$$P(\forall j\ \forall k \le n_j\ I_{j,k}^{(i)} = 0 \mid \exists j : \theta_j^{(i)} = 1) \le 0.05.$$

Раскрывая вероятность, получаем:

$$\frac{\prod_{j=1}^{40} [1 - q_j + q_j(1 - p_j)^{n_j}] - \prod_{j=1}^{40} (1 - q_j)}{1 - \prod_{j=1}^{40} (1 - q_j)} \le 0.05$$

5. Постановка задачи оптимизации

Пусть c_j — стоимость одного запуска теста j. Минимизируем общую стоимость:

$$\boxed{\min_{n_1,\ldots,n_{40}} \sum_{j=1}^{40} c_j n_j}$$

при ограничении на риск:

$$\frac{\prod_{j=1}^{40} [1 - \hat{q}_j + \hat{q}_j(1 - \hat{p}_j)^{n_j}] - \prod_{j=1}^{40}(1 - \hat{q}_j)}{1 - \prod_{j=1}^{40}(1 - \hat{q}_j)} \leq 0.05$$

5. Постановка задачи оптимизации

Пусть c_j — стоимость одного запуска теста j. Минимизируем общую стоимость:

$$\boxed{\min_{n_1,\ldots,n_{40}} \sum_{j=1}^{40} c_j n_j}$$

при ограничении на риск:

$$\frac{\prod_{j=1}^{40} [1 - \hat{q}_j + \hat{q}_j(1 - \hat{p}_j)^{n_j}] - \prod_{j=1}^{40}(1 - \hat{q}_j)}{1 - \prod_{j=1}^{40}(1 - \hat{q}_j)} \leq 0.05$$

Число запусков каждого теста:

$$\boxed{n_j \in \{0, 1, 2, 3, 4, 5\}, \quad j = 1, \ldots, 40.}$$

9. Алгоритм решения

- Есть численный алгоритм в Питоне.

9. Алгоритм решения

- Есть численный алгоритм в Питоне.
- Он называется **MILP**,и быстро дает оптимальный ответ.

Сведение к задаче MILP

Проблема полного перебора

Всего 40 тестов и 6 вариантов числа запусков $\Longrightarrow$ $6^{40} \approx 1.34 \cdot 10^{31}$ комбинаций. Прямой перебор невозможен.

Сведение к задаче MILP

Проблема полного перебора

Всего 40 тестов и 6 вариантов числа запусков $\Longrightarrow$ $6^{40} \approx 1.34 \cdot 10^{31}$ комбинаций. Прямой перебор невозможен.

Преобразуем нелинейное ограничение вероятности $(R(\mathbf{n}) \leq 0.05)$ с помощью логарифмирования в линейную сумму:

Сведение к задаче MILP

Проблема полного перебора

Всего 40 тестов и 6 вариантов числа запусков $\Longrightarrow$ $6^{40} \approx 1.34 \cdot 10^{31}$ комбинаций. Прямой перебор невозможен.

Преобразуем нелинейное ограничение вероятности $(R(\mathbf{n}) \leq 0.05)$ с помощью логарифмирования в линейную сумму:

$$\sum_{j=1}^{40} \underbrace{-\ln(1 - q_j + q_j(1 - p_j)^{n_j})}_{f_j(n_j)} \geq T$$

Сведение к задаче MILP

Проблема полного перебора

Всего 40 тестов и 6 вариантов числа запусков $\Longrightarrow$ $6^{40} \approx 1.34 \cdot 10^{31}$ комбинаций. Прямой перебор невозможен.

Преобразуем нелинейное ограничение вероятности $(R(\mathbf{n}) \leq 0.05)$ с помощью логарифмирования в линейную сумму:

$$\sum_{j=1}^{40} \underbrace{-\ln(1 - q_j + q_j(1 - p_j)^{n_j})}_{f_j(n_j)} \geq T$$

Вводим бинарные переменные $x_{jk} \in \{0, 1\}$. Количество запусков $n_j = \sum_{k=1}^{5} x_{jk}$ (при условии монотонности $x_{j,k+1} \leq x_{jk}$).

Сведение к задаче MILP

Проблема полного перебора

Всего 40 тестов и 6 вариантов числа запусков $\Longrightarrow$ $6^{40} \approx 1.34 \cdot 10^{31}$ комбинаций. Прямой перебор невозможен.

Преобразуем нелинейное ограничение вероятности $(R(\mathbf{n}) \leq 0.05)$ с помощью логарифмирования в линейную сумму:

$$\sum_{j=1}^{40} \underbrace{-\ln(1 - q_j + q_j(1 - p_j)^{n_j})}_{f_j(n_j)} \geq T$$

Вводим бинарные переменные $x_{jk} \in \{0, 1\}$. Количество запусков $n_j = \sum_{k=1}^{5} x_{jk}$ (при условии монотонности $x_{j,k+1} \leq x_{jk}$).
Обозначив $\Delta_{jk} = f_j(k) - f_j(k-1)$ как «пользу» от k-го запуска, получаем итоговую линейную задачу:

Сведение к задаче MILP

Проблема полного перебора

Всего 40 тестов и 6 вариантов числа запусков $\Longrightarrow$ $6^{40} \approx 1.34 \cdot 10^{31}$ комбинаций. Прямой перебор невозможен.

Преобразуем нелинейное ограничение вероятности $(R(\mathbf{n}) \leq 0.05)$ с помощью логарифмирования в линейную сумму:

$$\sum_{j=1}^{40} \underbrace{-\ln(1 - q_j + q_j(1 - p_j)^{n_j})}_{f_j(n_j)} \geq T$$

Вводим бинарные переменные $x_{jk} \in \{0, 1\}$. Количество запусков $n_j = \sum_{k=1}^{5} x_{jk}$ (при условии монотонности $x_{j,k+1} \leq x_{jk}$).
Обозначив $\Delta_{jk} = f_j(k) - f_j(k-1)$ как «пользу» от k-го запуска, получаем итоговую линейную задачу:

$$\min_{\mathbf{x}} \sum_{j,k} c_j x_{jk} \quad \text{при ограничении} \quad \sum_{j,k} \Delta_{jk} x_{jk} \geq T$$

Алгоритм поиска оптимума (Branch-and-Bound)

Для решения используется библиотека `scipy.optimize.milp` (солвер **HiGHS**).

Вместо слепого перебора применяется метод **ветвей и границ**:

- **Релаксация:** Временно разрешаем переменным быть дробными ($0 \leq x_{jk} \leq 1$). Решение такой задачи происходит очень быстро и дает оценку минимума снизу.

Алгоритм поиска оптимума (Branch-and-Bound)

Для решения используется библиотека `scipy.optimize.milp` (солвер **HiGHS**).

Вместо слепого перебора применяется метод **ветвей и границ**:

- **Релаксация:** Временно разрешаем переменным быть дробными ($0 \leq x_{jk} \leq 1$). Решение такой задачи происходит очень быстро и дает оценку минимума снизу.
- **Проверка:** Если оптимальное решение состоит только из 0 и 1 — глобальный оптимум найден.

Алгоритм поиска оптимума (Branch-and-Bound)

Для решения используется библиотека `scipy.optimize.milp` (солвер **HiGHS**).

Вместо слепого перебора применяется метод **ветвей и границ**:

- **Релаксация:** Временно разрешаем переменным быть дробными ($0 \leq x_{jk} \leq 1$). Решение такой задачи происходит очень быстро и дает оценку минимума снизу.
- **Проверка:** Если оптимальное решение состоит только из 0 и 1 — глобальный оптимум найден.
- **Ветвление (Branch):** Если переменная дробная (например, $x_{12} = 0.4$), задача разбивается на две независимые ветви: $x_{12} = 0$ и $x_{12} = 1$.

Алгоритм поиска оптимума (Branch-and-Bound)

Для решения используется библиотека `scipy.optimize.milp` (солвер **HiGHS**).

Вместо слепого перебора применяется метод **ветвей и границ**:

- **Релаксация:** Временно разрешаем переменным быть дробными ($0 \leq x_{jk} \leq 1$). Решение такой задачи происходит очень быстро и дает оценку минимума снизу.
- **Проверка:** Если оптимальное решение состоит только из 0 и 1 — глобальный оптимум найден.
- **Ветвление (Branch):** Если переменная дробная (например, $x_{12} = 0.4$), задача разбивается на две независимые ветви: $x_{12} = 0$ и $x_{12} = 1$.
- **Отсечение (Bound):** Если минимально возможная стоимость текущей ветви (из релаксации) уже хуже лучшего найденного решения, вся ветвь отбрасывается.

Валидация: Непараметрический бутстреп

Для валидации результатов мы применили непараметрический бутстреп:

- Генерировали по 70 релизов из исходной выборки с возвращением.

Валидация: Непараметрический бутстреп

Для валидации результатов мы применили непараметрический бутстреп:

- Генерировали по 70 релизов из исходной выборки с возвращением.
- Переоценивали параметры $\hat{p}_j$ и $\hat{q}_j$ на новых данных.

Валидация: Непараметрический бутстреп

Для валидации результатов мы применили непараметрический бутстреп:

- Генерировали по 70 релизов из исходной выборки с возвращением.
- Переоценивали параметры $\hat{p}_j$ и $\hat{q}_j$ на новых данных.
- Подставляли их в итоговую формулу вероятности пропуска ошибки.

Валидация: Непараметрический бутстреп

Для валидации результатов мы применили непараметрический бутстреп:

- Генерировали по 70 релизов из исходной выборки с возвращением.
- Переоценивали параметры $\hat{p}_j$ и $\hat{q}_j$ на новых данных.
- Подставляли их в итоговую формулу вероятности пропуска ошибки.

Валидация: Непараметрический бутстреп

Для валидации результатов мы применили непараметрический бутстреп:

- Генерировали по 70 релизов из исходной выборки с возвращением.
- Переоценивали параметры $\hat{p}_j$ и $\hat{q}_j$ на новых данных.
- Подставляли их в итоговую формулу вероятности пропуска ошибки.

Проблема: нестабильность оценок

Мы посчитали, в скольких случаях вероятность превышала целевые 0.05.

Оказалось, что очень часто она оказывалась **больше 0.2**!

Причина: переобучение на малых размерах выборок.

Анализ проблемы переобучения

Попробовали обойти проблему с переобучением с помощью кросс-валидации.

Анализ проблемы переобучения

Попробовали обойти проблему с переобучением с помощью кросс-валидации.

Делили датасет на 5 частей: обучались на 4 и тестировали на одной.

Анализ проблемы переобучения

Попробовали обойти проблему с переобучением с помощью кросс-валидации.

Делили датасет на 5 частей: обучались на 4 и тестировали на одной.

Оказалось, что это не спасает от малых размеров выборок, результаты всё ещё были нестабильны.

Решение: Параметрический бутстреп

Чтобы обойти проблему малых выборок, мы перешли к **параметрическому бутстрепу**.

Решение: Параметрический бутстреп

Чтобы обойти проблему малых выборок, мы перешли к **параметрическому бутстрепу**.

Алгоритм симуляции

- Используем $\hat{p}_j$ и $\hat{q}_j$, оцененные на оригинальных данных.

Решение: Параметрический бутстреп

Чтобы обойти проблему малых выборок, мы перешли к **параметрическому бутстрепу**.

Алгоритм симуляции

- Используем $\hat{p}_j$ и $\hat{q}_j$, оцененные на оригинальных данных.
- На их основе *генерируем* синтетические скрытые состояния $\theta_j^{(i)}$ и результаты тестов $I_{jk}^{(i)}$.

Решение: Параметрический бутстреп

Чтобы обойти проблему малых выборок, мы перешли к **параметрическому бутстрепу**.

Алгоритм симуляции

- Используем $\hat{p}_j$ и $\hat{q}_j$, оцененные на оригинальных данных.
- На их основе *генерируем* синтетические скрытые состояния $\theta_j^{(i)}$ и результаты тестов $I_{jk}^{(i)}$.
- Затем считаем, в скольких случаях мы пропустили ошибку:

Решение: Параметрический бутстреп

Чтобы обойти проблему малых выборок, мы перешли к **параметрическому бутстрепу**.

Алгоритм симуляции

- Используем $\hat{p}_j$ и $\hat{q}_j$, оцененные на оригинальных данных.
- На их основе *генерируем* синтетические скрытые состояния $\theta_j^{(i)}$ и результаты тестов $I_{jk}^{(i)}$.
- Затем считаем, в скольких случаях мы пропустили ошибку:

Решение: Параметрический бутстреп

Чтобы обойти проблему малых выборок, мы перешли к **параметрическому бутстрепу**.

Алгоритм симуляции

- Используем $\hat{p}_j$ и $\hat{q}_j$, оцененные на оригинальных данных.
- На их основе *генерируем* синтетические скрытые состояния $\theta_j^{(i)}$ и результаты тестов $I_{jk}^{(i)}$.
- Затем считаем, в скольких случаях мы пропустили ошибку:

$$\exists j : \max_{k \le n_j} I_{jk}^{(i)} \neq \theta_j^{(i)}$$

Решение: Параметрический бутстреп

Чтобы обойти проблему малых выборок, мы перешли к **параметрическому бутстрепу**.

Алгоритм симуляции

- Используем $\hat{p}_j$ и $\hat{q}_j$, оцененные на оригинальных данных.
- На их основе *генерируем* синтетические скрытые состояния $\theta_j^{(i)}$ и результаты тестов $I_{jk}^{(i)}$.
- Затем считаем, в скольких случаях мы пропустили ошибку:

$$\exists j : \max_{k \le n_j} I_{jk}^{(i)} \neq \theta_j^{(i)}$$

(То есть реальная проблема была ($\theta = 1$), но ни один из n_j запущенных тестов её не нашел).

Финальные результаты оптимизации

Прогнав симуляцию, мы сравнили изначальный подход (запускать всё по максимуму) и наш оптимизированный план.

Финальные результаты оптимизации

Прогнав симуляцию, мы сравнили изначальный подход (запускать всё по максимуму) и наш оптимизированный план.

Оригинальное решение

Если запускать все тесты максимальное число раз ($n_1 = \cdots = n_{40} = 5$), реальная вероятность ошибки составляет **3%**.

Финальные результаты оптимизации

Прогнав симуляцию, мы сравнили изначальный подход (запускать всё по максимуму) и наш оптимизированный план.

Оригинальное решение

Если запускать все тесты максимальное число раз ($n_1 = \cdots = n_{40} = 5$), реальная вероятность ошибки составляет **3%**.

Наш оптимизированный вариант

Мы настроили алгоритм так, чтобы целевая вероятность ошибки составляла **4,5%** (экономя ресурсы на запусках).

Финальные результаты оптимизации

Прогнав симуляцию, мы сравнили изначальный подход (запускать всё по максимуму) и наш оптимизированный план.

Оригинальное решение

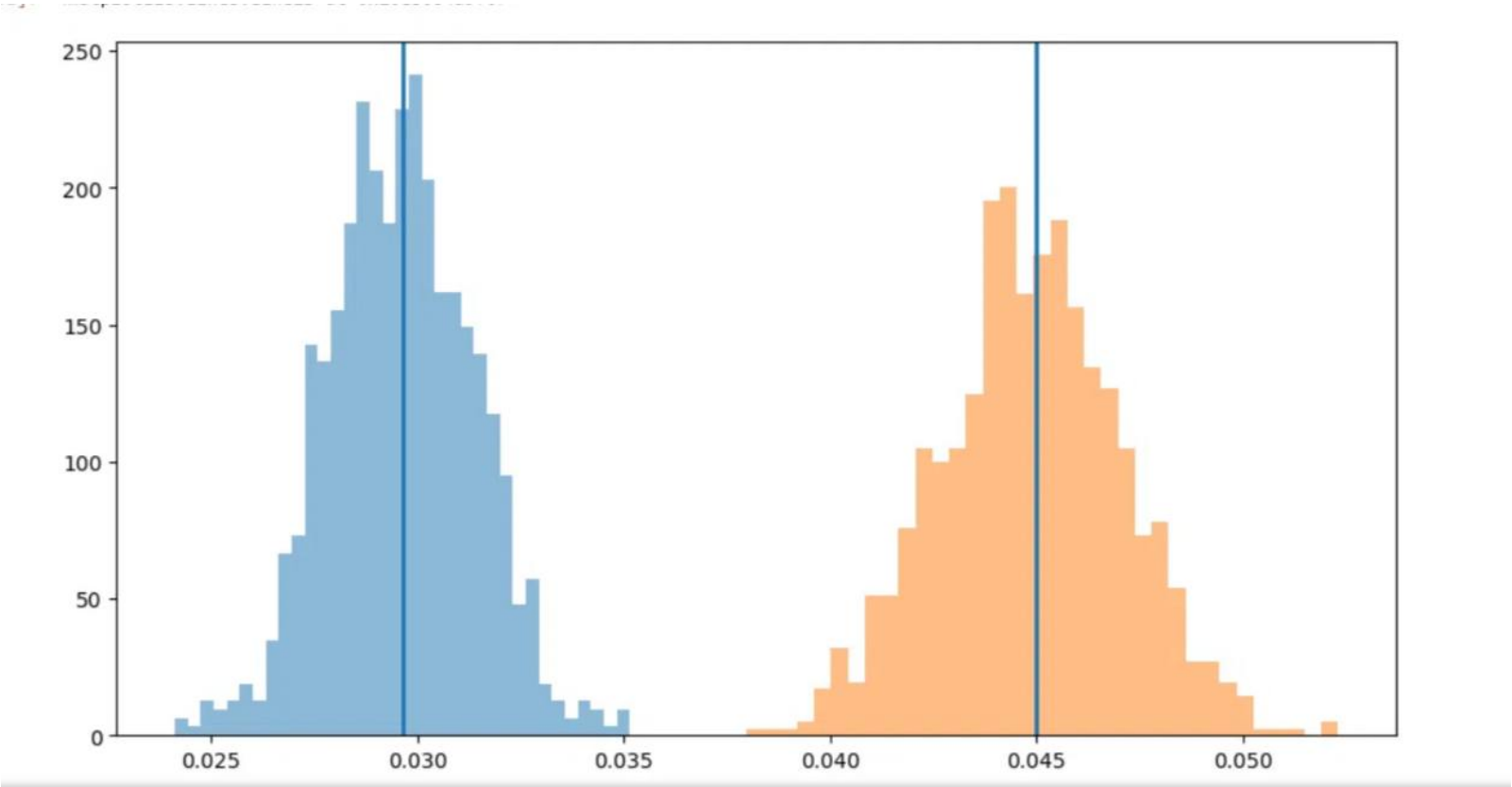
Если запускать все тесты максимальное число раз ($n_1 = \cdots = n_{40} = 5$), реальная вероятность ошибки составляет **3%**.

Наш оптимизированный вариант

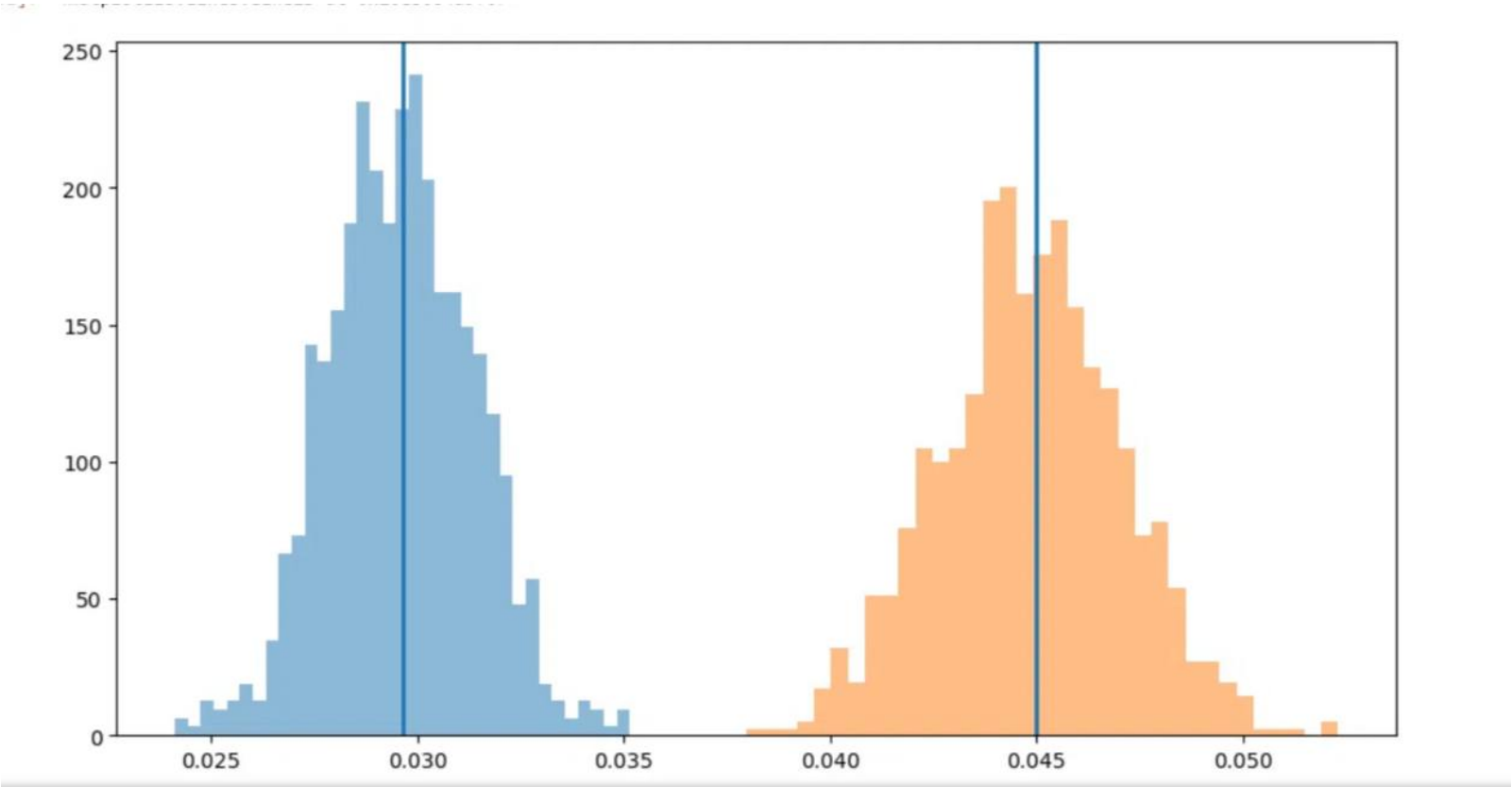
Мы настроили алгоритм так, чтобы целевая вероятность ошибки составляла **4,5%** (экономя ресурсы на запусках).

По результатам бутстрепа, наш вариант **в 97% случаев не превышает** строгий бизнес-лимит в 5%!

Распределение вероятности ошибки (Бутстреп)

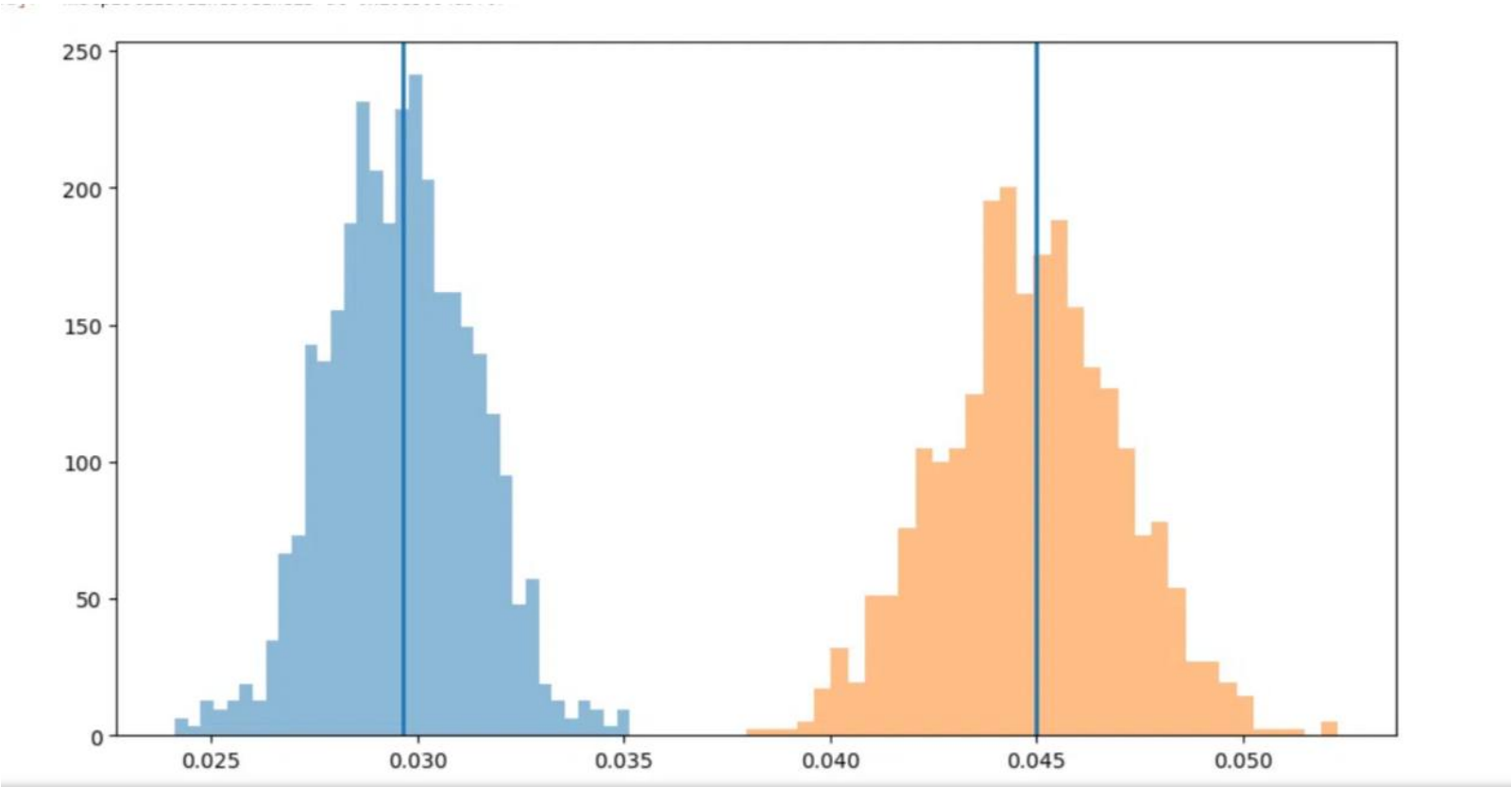


Распределение вероятности ошибки (Бутстреп)

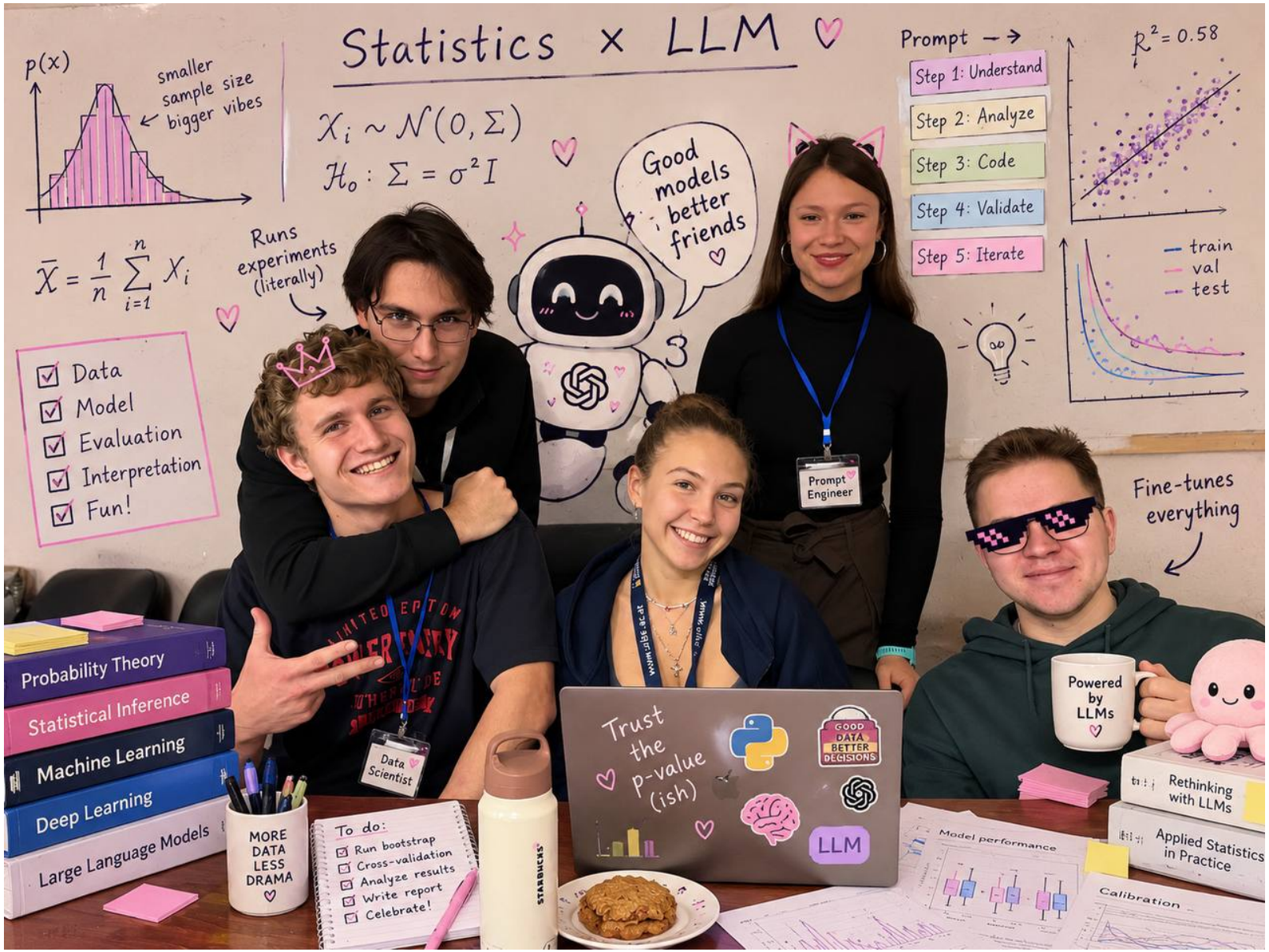


- **Синяя гистограмма:** Оригинальное решение ($n_j = 5$). Центрировано вокруг 0.030 (3%).

Распределение вероятности ошибки (Бутстреп)



- **Синяя гистограмма:** Оригинальное решение ($n_j = 5$). Центрировано вокруг 0.030 (3%).
- **Оранжевая гистограмма:** Наш оптимизированный план. Центрировано на 0.045 (4.5%). Правый хвост распределения практически не заходит за критическую отметку 0.05 (5%).


Statistics × LLM
$p(x)$
smaller sample size bigger vibes
$X_i \sim \mathcal{N}(0, \Sigma)$
$\mathcal{H}_0 : \Sigma = \sigma^2 I$
$\bar{X} = \frac{1}{n} \sum_{i=1}^{n} X_i$
Runs experiments (literally)
Good models better friends
Prompt →
Step 1: Understand
Step 2: Analyze
Step 3: Code
Step 4: Validate
Step 5: Iterate
$R^2 = 0.58$
train
val
test
Data
Model
Evaluation
Interpretation
Fun!
Prompt Engineer
Fine-tunes everything
Probability Theory
Statistical Inference
Machine Learning
Deep Learning
Large Language Models
MORE DATA LESS DRAMA
To do:
Run bootstrap
Cross-validation
Analyze results
Write report
Celebrate!
Data Scientist
Trust the p-value (ish)
GOOD DATA BETTER DECISIONS
LLM
Powered by LLMs
Rethinking with LLMs
Applied Statistics in Practice
Model performance
Calibration